

SISTEMAS DISTRIBUÍDOS TALLER 4

PROYECTO FINAL - PARTE I

OBJETIVO: Desplegar la primera parte del proyecto final de curso, evaluando conocimientos técnicos en manejo de sesiones, HTTP y WS.

El proyecto final del curso es un videojuego web multijugador donde varios jugadores se conectan desde sus navegadores y todos ven en tiempo real la posición y el nombre de los demás. Para llegar allá necesitamos cuatro piezas:

- Identidad (saber quién es cada jugador)
- Comunicación en vivo (mover la posición)
- Replicación (que el sistema aguante varios coordinadores)
- Tolerancia a fallos.

Este taller es la primera pieza: identidad. Sin login no hay jugador con nombre, y sin un canal autenticado no podemos confiar en lo que llega por WebSocket.

1. Qué se necesita

Vamos a construir tres componentes que corren como procesos separados y se comunican por red:

- Servicio de Autenticación (HTTP, puerto 4000): Registra usuarios, valida credenciales, emite tokens JWT. Tiene su propia base de datos SQLite.
- Coordinador de Sesiones (HTTP + WebSocket, puerto 5000): Recibe conexiones WS de los jugadores, valida el token JWT, mantiene en memoria la lista de jugadores conectados, y hace broadcast de esa lista cada vez que cambia.
- Cliente Web (HTML + JS plano, servido desde donde sea): Formulario de registro/login que habla con el servicio de auth, y una vez logueado abre el WS al coordinador y muestra en pantalla la lista de jugadores en línea.

Los tres componentes deben quedar desplegados con ngrok. Cada servicio en su propio túnel.

2. Requisitos Funcionales

Servicio de Auth

- **POST /register** recibe `{ username, password }` en JSON. Valida que `username` no exista. Hashea la contraseña con `bcrypt` (mínimo 10 rounds). Inserta en SQLite. Responde **201** con `{ userId, username }` o **409** si ya existe.
- **POST /login** recibe `{ username, password }`. Verifica contra la base. Si es válido, emite un JWT que contenga al menos `{ userId, username, iat, exp }` con expiración de 1 hora. Responde **200** con `{ token, username }` o **401** si las credenciales fallan.
- Manejo de errores correcto: **400** para body inválido, mensajes claros sin filtrar información sensible.
- La contraseña nunca sale de la base ni aparece en los logs.

Coordinador

- Acepta conexiones WS en `/connect` con el token como query param: `ws://host/connect?token=XXX`.
- Antes de aceptar la conexión, verifica el JWT con la misma secret que usó auth. Si está vencido, malformado o firmado con otra clave, cierra el socket inmediatamente con código **4001** y mensaje `"invalid token"`.
- Si el token es válido, agrega al usuario a un `Map` en memoria con la forma `userId → { username, socket, connectedAt }`.
- Cada vez que entra o sale un jugador, hace broadcast a todos los conectados con un mensaje JSON:

```
{ "type": "players_update", "players": [{"userId": "...", "username": "..."}, ...] }
```
- Detecta desconexiones (evento `close` del socket) y elimina al jugador del `Map` antes de hacer broadcast.
- Si el mismo usuario abre dos pestañas, decide qué hacer y documéntalo en el README: ¿rechazas la segunda conexión? ¿desconectas la primera? Cualquiera es válida, pero tienes que justificarla.

Cliente Web

- Página de login/registro con dos formularios. Al hacer login exitoso, guarda el token en `localStorage` y redirige a la pantalla del lobby.
- Pantalla del lobby: muestra el username del usuario actual y una lista de "Jugadores en línea" que se actualiza en vivo. Cuando alguien entra o sale, todos los clientes lo ven sin recargar.
- Botón de "Cerrar sesión" que borra el token y cierra el WS.
- Si el WS se cierra inesperadamente (token vencido, coordinador caído), muestra un mensaje al usuario y vuelve a la pantalla de login.

3. Requisitos No Funcionales

- **Estructura del repo:** Una carpeta `auth-service/`, una `coordinator/`, una `client/`. Cada servicio con su propio `package.json`.
- **Variables de entorno:** La secret de JWT, los puertos y las URLs no van hardcoded. Usa un archivo `.env` por servicio (con `dotenv`) y un `.env.example` versionado en el repo.
- **README:** El diagrama de la arquitectura, instrucciones para correr cada servicio paso a paso, ejemplos de `curl` para `/register` y `/login`, decisiones de diseño justificadas, y los comandos de ngrok que usaste.
- `.gitignore` que excluya `node_modules/`, `*.db`, `.env`.
- **Commits incrementales:** No hagas un único commit el martes a las 11pm. Vamos a mirar el historial.

4. Cómo arrancar

Paso 0: Estructura inicial

```
Shell
mkdir taller1-auth && cd taller1-auth
git init
mkdir auth-service coordinator client
```

En `auth-service/` y `coordinator/` corre `npm init -y` y luego instala las dependencias que necesitas en cada uno.

Paso 1: Auth service

Dependencias sugeridas: `express`, `better-sqlite3` (más simple que `sqlite3`, API síncrona), `bcrypt`, `jsonwebtoken`, `dotenv`.

Esqueleto de `auth-service/index.js`:

```
JavaScript
require('dotenv').config();
const express = require('express');
const Database = require('better-sqlite3');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

const db = new Database('users.db');
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
  )
`);

const app = express();
app.use(express.json());

app.post('/register', async (req, res) => {
  // TODO: validar body, hashear, insertar, manejar duplicado
});
```

```
app.post('/login', async (req, res) => {  
  // TODO: buscar usuario, comparar hash, emitir JWT con expiración 1h  
});  
  
app.listen(process.env.PORT || 4000, () => {  
  console.log(`Auth service en puerto ${process.env.PORT || 4000}`);  
});
```

No avances hasta que esto funcione (o probar en Postman / Thunder Client):

```
Shell  
curl -X POST http://localhost:4000/register \  
  -H "Content-Type: application/json" \  
  -d '{"username":"alice","password":"secret123"}'  
  
curl -X POST http://localhost:4000/login \  
  -H "Content-Type: application/json" \  
  -d '{"username":"alice","password":"secret123"}'  
  
# Debe devolver { "token": "eyJhbGc...", "username": "alice" }
```

Paso 2: Coordinador

Dependencias sugeridas: `express`, `ws`, `jsonwebtoken`, `dotenv`. La misma `JWT_SECRET` que en auth. Esqueleto de `coordinator/index.js`:

```
JavaScript  
require('dotenv').config();  
const express = require('express');  
const http = require('http');  
const { WebSocketServer } = require('ws');  
const jwt = require('jsonwebtoken');  
const url = require('url');  
  
const app = express();  
const server = http.createServer(app);  
const wss = new WebSocketServer({ noServer: true });
```

```
const players = new Map(); // userId -> { username, socket, connectedAt
}

function broadcastPlayers() {
  const list = Array.from(players.entries()).map(([userId, p]) => ({
    userId, username: p.username
  }));
  const msg = JSON.stringify({ type: 'players_update', players: list
});
  for (const { socket } of players.values()) {
    if (socket.readyState === socket.OPEN) socket.send(msg);
  }
}

server.on('upgrade', (req, socket, head) => {
  const { query } = url.parse(req.url, true);
  const token = query.token;
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    wss.handleUpgrade(req, socket, head, (ws) => {
      wss.emit('connection', ws, payload);
    });
  } catch (err) {
    socket.write('HTTP/1.1 401 Unauthorized\r\n\r\n');
    socket.destroy();
  }
});

wss.on('connection', (ws, payload) => {
  // TODO: agregar a players, broadcast, manejar 'close'
});

server.listen(process.env.PORT || 5000);
```

Prueba con **wscat** antes de tocar el cliente:

```
Shell
npm install -g wscat
wscat -c "ws://localhost:5000/connect?token=PEGA_AQUI_EL_TOKEN"
# En otra terminal, conecta otro usuario y deberías ver el broadcast
```

Paso 3: Cliente

Dos páginas HTML simples, JS plano. No te metas con frameworks, no es el punto del taller. Sirve los archivos con `npm serve client` o con un Express minúsculo en el puerto 3000. Esqueleto de la conexión WS en el cliente:

JavaScript

```
const token = localStorage.getItem('token');
const COORDINATOR_WS = 'ws://localhost:5000';
const ws = new WebSocket(`${COORDINATOR_WS}/connect?token=${token}`);

ws.onmessage = (event) => {
  const msg = JSON.parse(event.data);
  if (msg.type === 'players_update') {
    renderPlayers(msg.players);
  }
};

ws.onclose = () => {
  alert('Sesión terminada');
  localStorage.removeItem('token');
  window.location.href = 'login.html';
};
```

Paso 4: Ngrok

Abre tres terminales:

Shell

Terminal 1

```
ngrok http 4000 # https://abc123.ngrok-free.app -> auth
```

Terminal 2

```
ngrok http 5000 # Da otra URL -> coordinador
```

Terminal 3

```
ngrok http 3000 # Da otra URL -> cliente
```

Importante: Cuando expongas el coordinador con ngrok, el cliente debe usar `wss://` (no `ws://`), porque ngrok te da HTTPS. Y la URL del cliente al servicio de auth también debe ser la `https://` de ngrok.

5. Errores típicos

1. **Tener un solo puerto para todo.** No. Auth y coordinador son procesos distintos, en puertos distintos.
2. **Hardcodear la secret de JWT.** Va en `.env`, y la misma en los dos servicios. Si son distintas, los tokens no validan.
3. **No verificar el token en el `upgrade`.** Si validas dentro de `wss.on('connection')`, el socket ya está abierto cuando rechazas. Funciona, pero es una vulnerabilidad menor y se nota en código. Hazlo en el `upgrade`.
4. **Hacer broadcast solo al que se conecta.** El broadcast va a *todos* los conectados, incluyendo al nuevo.
5. **No manejar el `close`.** Si un usuario cierra la pestaña y no lo eliminas del `Map`, sigue apareciendo "en línea" para siempre.
6. **Guardar la contraseña en plano "solo para probar".** No. Bcrypt desde el primer commit.
7. **Mezclar `ws://` con `https://` o viceversa en ngrok.** El navegador bloquea contenido mixto. Si la página la sirves por HTTPS, el WebSocket también tiene que ser `wss://`.
8. **CORS.** Si el cliente y el auth están en orígenes distintos, vas a necesitar `cors()` en el auth. Instálalo y úsalo: `app.use(cors())`.

6. Entrega y sustentación

Qué se entrega:

1. **Link al repo de GitHub.** Commits incrementales, no uno solo.
2. **README completo.**
3. **Tres URLs de ngrok** activas durante tu turno de sustentación.

Qué se sustenta:

1. Demuestra el flujo: Registro de un usuario nuevo, login, conexión al lobby. Abre dos navegadores distintos y muestra que ambos se ven entrar y salir en vivo (1 min).
2. Muestra qué pasa con un token inválido: Edita el `localStorage`, ensucia el token, recarga, demuestra que el coordinador rechaza la conexión (1 min).
3. Muestra qué pasa con un token vencido: Se te pedirá que cambies la expiración a 30 segundos en vivo y reproduzcas el caso (2 min).
4. Pasea por el código: Estructura del repo, cómo verifica el coordinador el JWT, cómo manejas el broadcast (3 min).
5. Responde preguntas (3 min).